Fork Progress Report

CaseMirror Research

casemirror.ai

June 6, 2026

Project: sia_rust

Repository: /home/m/sia_rust

Survey date: 2026-06-06

Survey HEAD: da4191a on main

A. Executive Summary

Since the fork from the upstream SIA project, this repository has moved from a

Python-first self-improving-agent framework into an active Rust implementation

with a Python execution bridge. The deterministic SIA loop now has Rust-native

configuration, profile loading, prompt construction, context tracking,

orchestration scaffolding, web visualization, benchmarks, parity checks, and a

standalone evaluation crate.

The fork-era work spans 48 commits after the inferred upstream baseline 4b4877f,

with 139 changed files, 33,895 insertions, and 333 deletions. Most of the work is

in the Rust source tree, the optional native LLM layer, tests, documentation,

benchmarks, evaluation support, and Beads issue tracking.

The current offline verification posture is strong. The default Rust test suite

passes, formatting and clippy pass for both default and LLM-feature builds, the

evaluation crate passes, and the Python/Rust parity script reports byte-identical

outputs. The main remaining risk is live readiness: provider credentials, ignored

live tests, demo safety, dashboard freshness, and closed-loop actuation still need

focused work.

B. Scope And Baseline

This checkout has no configured upstream remote. The only configured remote is

origin, pointing at the Micah Stubbs GitHub fork. For that reason this report uses

4b4877f, titled "chore: bump minor version as cli contracts have changed (#23)",

as the inferred upstream baseline.

There were 11 commits through that baseline and 48 fork-era commits after it at

the time of survey. The surveyed local head was da4191a, "Sync beads tracker after

upstream review." During PDF generation, origin/main advanced by three additional

remote commits, so this report evaluates the local checkout at da4191a and does

not cover those later remote commits. The report/PDF artifact commit itself is

also outside the survey counts.

C. Work Completed

1. Rust Port Foundation

The largest milestone is 49415c2, "Rust port of SIA framework (exhaustive parity +

benchmarks + dsrs evals)." It introduced the Rust crate, command-line binary,

CI workflow, Criterion benchmarks, parity tooling, and the core ported modules.

The Rust code now mirrors the Python package surface for configuration, providers,

profiles, task layout, run setup, prompt building, context management,

orchestration, results, and web data.

The target-agent execution boundary intentionally remains Python. Generated target

agents still run as Python subprocesses through the existing evaluate.py task

contract, preserving the original paper's task/evaluator semantics while allowing

the orchestrator and deterministic scaffolding to move into Rust.

The project framing was rewritten accordingly. The README and Rust-port

documentation now present the repository as a Rust implementation with a Python

bridge, and docs/RUST_PORT.md maps the Python modules to their Rust equivalents.

2. Native LLM Layer

The fork added a substantial optional native LLM layer under src/llm, gated by the

LLM Cargo feature. It includes an AgentRunner abstraction, Anthropic Messages API

types, OpenAI-compatible chat transport types, Claude/OpenHands/PydanticAI-style

runner loops, tool execution, retry/backoff behavior, structured-output support,

trajectory logging, telemetry, provider-to-client mapping, and a Tavily search

client primitive.

This closes a major boundary from the initial port. Meta and feedback agents can

now use native Rust LLM clients when built with the LLM feature, while the target

agent continues to use the Python task bridge.

3. Closed-Loop Research Extensions

The fork added first-pass machinery for research extensions beyond plain harness

updates. The scheduler records adaptive decisions and exposes them through both

artifacts and the web dashboard. The weights module defines a native update

abstraction and a CPU reference LoRA-style updater. The verifier module adds

reusable exact-match, multiple-choice, numeric, and robustness helpers.

These extensions are useful but not yet complete. The scheduler decision is still

observational in important places, and the CPU weight path is a reference

demonstrator rather than a real model-training path whose adapter is loaded by a

target agent.

4. SIA Studio

The web surface has been upgraded into a file-backed "SIA Studio" dashboard with

telemetry, metrics charts, dark-mode polish, scheduler timelines, and endpoints

for telemetry, metrics, scheduler decisions, and weight-update artifacts. It is

already useful for replaying and inspecting runs, but open issues correctly flag

live-demo gaps: the UI needs auto-refresh, dashboard port binding needs clearer

failure/fallback behavior, and demo run IDs need collision-proof defaults.

5. Safety And Sandboxing

The fork added a formal threat model in SECURITY.md and a first auditable

capability layer in src/sandbox.rs. Native tool executors can check read, write,

bash, path-containment, and size permissions. The orchestrator also has Docker

sandbox command generation for Python target-agent execution.

The safety story is honest but incomplete. The default native-runner capability

profile still permits broad Bash behavior, and Docker mode is not yet a drop-in

safe path for live provider-calling target agents because network access,

dependencies, and credential handling require a clearer policy.

6. Providers And Reproducibility

The fork added Nebius and Gemini support, later expanded with a Tinker provider

and bundled GPT-OSS and Qwen3 Tinker target profiles. It also added .env loading,

credential documentation, Nebius quickstarts, model-slug verification tooling,

reproducibility standards, a hackathon run-of-show, a slide outline, and a

preprint-style project paper draft.

The project now has a credible external-facing package for a demo or research

submission. The remaining weakness is not documentation volume; it is live

validation with real provider keys and recorded end-to-end artifacts.

7. Benchmarks And Evals

The fork added Criterion benchmarks, Python-vs-Rust benchmark scripts, a benchmark

report, a standalone DSRs/dspy-rs evaluation crate, and a differential parity

script. The benchmark report records the deterministic core as about 5.8x faster

by geometric mean, while the parity script confirms byte-identical behavior across

JSON, prompt, feedback-context, and execution-log loading surfaces.

8. Operations

The project now has Beads tracking, local agent guidance, a package manifest,

GitHub-issue mirroring scripts, session summaries, close-session tracker lessons,

expanded CLAUDE.md instructions, and a packaging metadata guard for the OpenHands

optional extra. At survey time Beads showed 18 open issues and 17 ready issues.

The most urgent ready work is demo and safety oriented: hardened native-runner

capabilities, Docker-sandbox clarity, a stage-safe demo command, and SIA Studio

auto-refresh.

D. Verification

The default Rust suite passed with cargo test. The library crate reported 154

passed tests, and the integration and doc tests also passed.

The LLM-feature suite still has a parallel-test isolation bug. Under default

parallel execution, provider-mapping tests race on process environment variables;

that run aborted after 272 library tests passed, 4 failed, and 6 were ignored.

The same suite passed when serialized with one test thread: 276 library tests

passed, 6 live-provider tests remained ignored, and the LLM integration and doc

tests passed.

The standalone eval crate passed its 4 offline tests. Formatting passed, and

clippy passed with warnings denied for both the default and LLM-feature builds.

The Python/Rust parity script reported "PARITY OK: all surfaces byte-identical."

The broader Python pytest suite was not run because pytest is not installed in

this environment; the packaging metadata unittest passed.

E. Current State

Before updating this report and generating the PDF artifacts, the worktree was

clean except for the branch relationship: main was one commit ahead of origin/main.

During report/PDF generation, an unstaged Beads JSONL update appeared; it was left

outside the report artifact commit.

F. Remaining Risks

The next high-value work is to make the LLM-feature tests pass under normal

parallel execution, then run ignored live-provider tests with real Anthropic,

OpenHands-compatible, PydanticAI-compatible, Gemini, Nebius, structured-output,

and Tavily credentials. A complete live SIA run should be recorded with artifacts.

The closed-loop story also needs to become acting rather than merely visible:

scheduler decisions should drive the generation path, and the weight-update path

should either become a real model update or be clearly scoped as an offline

demonstrator. Demo hardening remains important: safer native-runner permissions,

Docker policy clarity, auto-refreshing dashboards, reliable port behavior,

collision-proof run IDs, and a fast stage-safe task.

G. Overall Assessment

The fork has already accomplished the hard architectural work. The deterministic

SIA loop is represented in Rust, parity is actively tested, native meta/feedback

LLM runners exist, telemetry and dashboard surfaces are in place, and the project

has a serious documentation and demo package.

The project is not yet live-demo hardened. Its deterministic and offline

verification story is strong, but live-provider execution, security defaults,

dashboard freshness, and closed-loop actuation still need focused work before the

repository can credibly claim a production-ready self-improving loop.